Project Execution Procedure - Cyberbullying Detection using DistilBERT

Step 1: Set Up the Environment

Install Python on your system (recommended: Python 3.8 or later). Set up a virtual environment (optional but preferred for clean project dependencies). Install all necessary libraries like transformers, pandas, scikit-learn, torch, and flask.

Step 2: Prepare the Dataset

Download the cyberbullying-labeled dataset (in CSV format). Clean the text data by removing unwanted elements like URLs, mentions, emojis, and punctuation. Perform data preprocessing and split the dataset into training and testing sets.

Step 3: Load the Pre-trained Model

Use the DistilBERT model from the Hugging Face library. Load the tokenizer and the classification model. Tokenize the cleaned text data to convert it into model-understandable format.

Step 4: Train the Model

Fine-tune the DistilBERT model on your training data. Monitor the models performance using metrics like accuracy, precision, recall, and F1-score. Use early stopping or learning rate scheduling to prevent overfitting.

Step 5: Save the Trained Model

After training, save the model and tokenizer locally. This allows reusing the model for predictions without retraining.

Step 6: Build the Web Interface (Flask App)

Create a basic Flask application with an input form. Load the saved model inside the Flask app. When a user submits a message, the app sends it to the model for classification. Display the result: whether the message is cyberbullying or not, along with the confidence score.

Step 7: Test the Application

Run the Flask application on your local server. Test the prediction system using different text inputs. Verify that the outputs are accurate and the interface is working smoothly.

Step 8: Optional Enhancements

Add a logging system to record classified texts. Add thresholding to flag uncertain predictions. Deploy the web app to a cloud platform like Heroku or Render for public access.

Step 9: Documentation and Results

Document all steps, results, graphs, and accuracy reports. Include comparison with other models like LSTM, BiLSTM, and BERT to show improvement. Prepare screenshots and logs from the working app.